Python Database Practicals

SQLite Practical Questions 1–7

Easy-to-follow Python code with short explanations and sample outputs.

Note: These examples use the database and table names from the practical questions. Run Practical 1 first, then continue in order. Practical 2 creates the tables required by Practicals 3–7.

Practical 1: Create SQLite Database

Question: Create a SQLite database named university_assignment.db.

```
import sqlite3

# Connect to database
conn = sqlite3.connect("university_assignment.db")

# Create cursor
cur = conn.cursor()

print("Database created successfully!")

# Close connection
conn.close()
```

Sample output:

```
Database created successfully!
```

Practical 2: Insert Sample Data Using executemany()

Question: Insert sample data into students, courses and enrollments using executemany() and commit the changes.

```
import sqlite3

conn = sqlite3.connect("university_assignment.db")
cur = conn.cursor()

# Enable foreign keys
cur.execute("PRAGMA foreign_keys = ON")

# Create tables
cur.execute("""
CREATE TABLE IF NOT EXISTS students(
    id INTEGER PRIMARY KEY,
    name TEXT,
    email TEXT,
    major TEXT
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS courses(
    course_id INTEGER PRIMARY KEY,
    course_name TEXT
)
""")

cur.execute("""
CREATE TABLE IF NOT EXISTS enrollments(
    enrollment_id INTEGER PRIMARY KEY,
    student_id INTEGER,
    course_id INTEGER,
    marks INTEGER,
    FOREIGN KEY(student_id) REFERENCES students(id),
    FOREIGN KEY(course_id) REFERENCES courses(course_id)
)
""")

# Insert students
students = [
    (1, "Rahul", "rahul@gmail.com", "Computer"),
    (2, "Priya", "priya@gmail.com", "Physics"),
    (3, "Amit", "amit@gmail.com", "Computer")
]

cur.executemany(
    "INSERT OR IGNORE INTO students VALUES (?, ?, ?, ?)",
    students
)

# Insert courses
courses = [
    (101, "Python"),
    (102, "Database"),
```

```
    (103, "Machine Learning")
]

cur.executemany(
    "INSERT OR IGNORE INTO courses VALUES (?, ?)",
    courses
)

# Insert enrollments
enrollments = [
    (1, 1, 101, 85),
    (2, 2, 102, 90),
    (3, 3, 103, 80)
]

cur.executemany(
    "INSERT OR IGNORE INTO enrollments VALUES (?, ?, ?, ?)",
    enrollments
)

conn.commit()
print("Data inserted successfully!")
conn.close()
```

Sample output:

```
Data inserted successfully!
```

Practical 3: Retrieve Data Using SQL JOIN

Question: Retrieve and display student, course and enrollment data using SQL queries and JOINs.

```
import sqlite3

conn = sqlite3.connect("university_assignment.db")
cur = conn.cursor()

query = """
SELECT students.name,
       courses.course_name,
       enrollments.marks
FROM enrollments
JOIN students
ON enrollments.student_id = students.id
JOIN courses
ON enrollments.course_id = courses.course_id
"""

cur.execute(query)
rows = cur.fetchall()

print("Student Records:")
for row in rows:
    print(row)

conn.close()
```

Sample output:

```
Student Records:
('Rahul', 'Python', 85)
('Priya', 'Database', 90)
('Amit', 'Machine Learning', 80)
```

Practical 4: Update Student and Enrollment Records

Question: Update a student name and the marks in an enrollment record, then commit the changes.

```
import sqlite3

conn = sqlite3.connect("university_assignment.db")
cur = conn.cursor()

# Update student name
cur.execute("""
UPDATE students
SET name = ?
```

```
WHERE id = ?
""", ("Rohit", 1))

# Update enrollment marks
cur.execute("""
UPDATE enrollments
SET marks = ?
WHERE student_id = ?
""", (95, 1))

conn.commit()
print("Records updated successfully!")
conn.close()
```

Sample output:

```
Records updated successfully!
```

Practical 5: Delete Records Using Foreign Key Constraints

Question: Enable foreign-key checking. Delete the student's enrollment first, then delete the student record.

```
import sqlite3

conn = sqlite3.connect("university_assignment.db")
cur = conn.cursor()

cur.execute("PRAGMA foreign_keys = ON")

try:
    # Delete enrollment first
    cur.execute("""
    DELETE FROM enrollments
    WHERE student_id = ?
    """, (2,))

    # Delete student
    cur.execute("""
    DELETE FROM students
    WHERE id = ?
    """, (2,))

    conn.commit()
    print("Student deleted successfully!")

except sqlite3.IntegrityError:
    conn.rollback()
    print("Foreign key constraint error!")

conn.close()
```

Sample output:

```
Student deleted successfully!
```

Practical 6: Create Full, Schema-only and Data-only Dumps

Question: Create three SQL files: a full database dump, a schema-only dump and a data-only dump.

```
import sqlite3

conn = sqlite3.connect("university_assignment.db")
cur = conn.cursor()

# Full database dump
with open("full_dump.sql", "w", encoding="utf-8") as f:
    for line in conn.iterdump():
        f.write(line + "\n")

# Get user-created tables
cur.execute("""
SELECT name, sql
FROM sqlite_master
WHERE type = 'table'
AND name NOT LIKE 'sqlite_%'
""")
tables = cur.fetchall()
```

```
# Schema-only dump
with open("schema_dump.sql", "w", encoding="utf-8") as f:
    for table_name, create_sql in tables:
        if create_sql:
            f.write(create_sql + ";\n\n")

# Data-only dump
with open("data_dump.sql", "w", encoding="utf-8") as f:
    for table_name, create_sql in tables:
        # Quote table name safely for SELECT
        quoted_name = '"' + table_name.replace('"', '""') + '"'
        cur.execute("SELECT * FROM " + quoted_name)
        rows = cur.fetchall()
        columns = [item[0] for item in cur.description]

        for row in rows:
            values = []
            for value in row:
                if value is None:
                    values.append("NULL")
                elif isinstance(value, (int, float)):
                    values.append(str(value))
                else:
                    escaped = str(value).replace("'", "''")
                    values.append("'" + escaped + "'")

            col_list = ", ".join('"' + c.replace('"', '""') + '"' for c in columns)
            f.write(
                "INSERT INTO " + quoted_name +
                " (" + col_list + ") VALUES (" +
                ", ".join(values) + ");\n"
            )

print("Full, schema-only and data-only dumps created!")
conn.close()
```

Sample output:

```
Full, schema-only and data-only dumps created!
```

Practical 7: Export Enrollment Data and Import Students from CSV

Question: Export enrollment records to a CSV file and import new student records from a CSV file.

```
import sqlite3
import csv

conn = sqlite3.connect("university_assignment.db")
cur = conn.cursor()

# Export enrollment data
cur.execute("SELECT * FROM enrollments")

with open("enrollments.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow([
        "enrollment_id", "student_id", "course_id", "marks"
    ])
    writer.writerows(cur.fetchall())

print("CSV exported successfully!")

# Create a sample CSV file for importing
with open("new_students.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "name", "email", "major"])
    writer.writerow([10, "Neha", "neha@gmail.com", "Computer"])

# Import student records
with open("new_students.csv", "r", newline="", encoding="utf-8") as f:
    reader = csv.DictReader(f)

    for row in reader:
        cur.execute("""
        INSERT INTO students(id, name, email, major)
        VALUES (?, ?, ?, ?)
        """, (
            row["id"], row["name"], row["email"], row["major"]
        ))
```

```
conn.commit()
print("Student CSV imported successfully!")
conn.close()
```

Sample output:

```
CSV exported successfully!
Student CSV imported successfully!
```